

Chiseling101

September 9, 2026

1 Chiseling 101

CC-BY 2026 Brooksbank, Kassabov, Wilson

Dleto (Chisel) detects and locates sparsity patterns in tensor data. This notebook demonstrates the basic functionality of Dleto, how to select different chisels, and what to expect from the outputs.

1. Tucker Decompositions
2. Block Decompositions
3. Stratification

1.1 1. Tucker Decompositions

We first use Dleto to find familiar tensor structures known as Tucker Decompositions. You can think of these as tensor generalizations of matrix nullspaces.

Performance. Tucker decompositions have been explored since the 1800's, and there are highly optimized strategies to discover them. This tutorial uses a familiar problem to explore the Dleto's functionality, but the performance of our demonstration is far from optimal on this particular application. Due to its generality, Dleto chiseling operates with a complexity slightly greater than more direct Tucker Decomposition strategies. Indeed Dleto itself leverages such strategies through its function `nondeg`.

A **Tucker Decomposition** of a tensor Γ framed by *axes* (known also as *modes*, *legs*, or *indices*) $\mathbb{K}^{d_1}, \dots, \mathbb{K}^{d_\ell}$, is a subset $A \subset \{1, \dots, \ell\}$ and a decomposition

$$\forall a \in A, \quad \mathbb{K}^{d_a} = E_a \oplus R_a$$

such that the contraction of Γ on R_a is 0.

The decompositions can be given by partitioned bases of E_a and R_a , or equivalently by linear projections $e_a : \mathbb{K}^{d_a} \rightarrow \mathbb{K}^{e_a}$ with kernel R_a . In many situations the purpose of a Tucker decomposition is to restrict the tensor to the E_a -spaces, which we call the “truncated” form.

1.1.1 1.1 Loading Dleto

Start by loading `Dleto.jl`. First time users may need to install auxiliary packages and possibly set up Julia for notebooks. That is a one-time setup for which we provide instructions. Alternatively, you can use the links on the landing page to access the online Binder demonstration.

1.1.2 !Colab users begin here!

While [Google's Colab](#) supports Julia, its default configuration is Python and you will need to invite Colab to use Julia before proceeding, which can be done either by selecting the Runtime or by running the following cell first.

```
[1]: # %% julia
```

```
using Pkg
Pkg.add( "IJulia" )
Pkg.add( "Plots" )
```

Resolving package versions...

```
version 6.10.2+1 of package Qt6Base_jll is not available. Available versions: 6
↳ 0.3+0, 6.0.3+1, 6.3.0+0, 6.3.0+1, 6.4.1+0, 6.4.1+1, 6.4.2+0, 6.4.2+1, 6.4.2+2
↳ 6.4.2+3, 6.5.2+0, 6.5.2+1, 6.5.2+2, 6.5.3+0, 6.5.3+1, 6.7.0+0, 6.7.0+1, 6.7.
↳ 1+0, 6.7.1+1, 6.8.1+0, 6.8.2+0, 6.8.2+1, 6.8.2+2, 6.10.2+2
```

Stacktrace:

```
[1] pkgerror(msg::String)
   @ Pkg.Types ~/.julia/juliaup/julia-1.12.3+0.aarch64.apple.darwin14/share/
↳ julia/stdlib/v1.12/Pkg/src/Types.jl:68
[2] resolve_versions!(env::Pkg.Types.EnvCache, registries::Vector{...}, pkgs::
↳ Vector{...}, julia_version::VersionNumber, installed_only::Bool)
   @ Pkg.Operations ~/.julia/juliaup/julia-1.12.3+0.aarch64.apple.darwin14/
↳ share/julia/stdlib/v1.12/Pkg/src/Operations.jl:598
[3] targeted_resolve
   @ ~/.julia/juliaup/julia-1.12.3+0.aarch64.apple.darwin14/share/julia/stdlib
↳ v1.12/Pkg/src/Operations.jl:1736 [inlined]
[4] tiered_resolve(env::Pkg.Types.EnvCache, registries::Vector{...}, pkgs::
↳ Vector{...}, julia_version::VersionNumber, try_all_installed::Bool)
   @ Pkg.Operations ~/.julia/juliaup/julia-1.12.3+0.aarch64.apple.darwin14/
↳ share/julia/stdlib/v1.12/Pkg/src/Operations.jl:1708
[5] _resolve(io::IOContext{...}, env::Pkg.Types.EnvCache, registries::Vector{...}
↳ pkgs::Vector{...}, preserve::Pkg.Types.PreserveLevel, julia_version::
↳ VersionNumber)
   @ Pkg.Operations ~/.julia/juliaup/julia-1.12.3+0.aarch64.apple.darwin14/
↳ share/julia/stdlib/v1.12/Pkg/src/Operations.jl:1748
[6] add(ctx::Pkg.Types.Context, pkgs::Vector{...}, new_git::Set{...};
↳ allow_autoprecomp::Bool, preserve::Pkg.Types.PreserveLevel, platform::Base.
↳ BinaryPlatforms.Platform, target::Symbol)
   @ Pkg.Operations ~/.julia/juliaup/julia-1.12.3+0.aarch64.apple.darwin14/
↳ share/julia/stdlib/v1.12/Pkg/src/Operations.jl:1783
[7] add
   @ ~/.julia/juliaup/julia-1.12.3+0.aarch64.apple.darwin14/share/julia/stdlib
↳ v1.12/Pkg/src/Operations.jl:1754 [inlined]
[8] add(ctx::Pkg.Types.Context, pkgs::Vector{...}; preserve::Pkg.Types.
↳ PreserveLevel, platform::Base.BinaryPlatforms.Platform, target::Symbol,
↳ allow_autoprecomp::Bool, kwargs::@Kwargs{...})
```

```

@ Pkg.API ~/.julia/juliaup/julia-1.12.3+0.aarch64.apple.darwin14/share/juli. /
↳stdlib/v1.12/Pkg/src/API.jl:334
[9] add(pkgs::Vector{Pkg.Types.PackageSpec}; io::IOContext{IO}, kwargs::
↳@Kwargs{})
@ Pkg.API ~/.julia/juliaup/julia-1.12.3+0.aarch64.apple.darwin14/share/juli. /
↳stdlib/v1.12/Pkg/src/API.jl:169
[10] add(pkgs::Vector{Pkg.Types.PackageSpec})
@ Pkg.API ~/.julia/juliaup/julia-1.12.3+0.aarch64.apple.darwin14/share/juli. /
↳stdlib/v1.12/Pkg/src/API.jl:158
[11] add(pkgs::Vector{String}; kwargs::@Kwargs{})
@ Pkg.API ~/.julia/juliaup/julia-1.12.3+0.aarch64.apple.darwin14/share/juli. /
↳stdlib/v1.12/Pkg/src/API.jl:157
[12] add
@ ~/.julia/juliaup/julia-1.12.3+0.aarch64.apple.darwin14/share/julia/stdlib.
↳v1.12/Pkg/src/API.jl:157 [inlined]
[13] add(pkg::String)
@ Pkg.API ~/.julia/juliaup/julia-1.12.3+0.aarch64.apple.darwin14/share/juli. /
↳stdlib/v1.12/Pkg/src/API.jl:156
[14] top-level scope
@ In[1]:3
[15] eval(m::Module, e::Any)
@ Core ./boot.jl:489
Some type information was truncated. Use `show(err)` to see complete types.

```

1.1.3 All other users begin here.

If you are running from your own Jupyter environment, make sure to have installed a Julia kernel (version 1.11 or higher) and configured Jupyter to use Julia.

```

[2]: # Uncomment and run the first time, if Dleto is not installed
using Pkg

# Option 1: To install from remote repository, use:
# Pkg.add(url="https://github.com/thetensor-space/OpenDleto")

# Option 2: If cloned locally at PATH
Pkg.activate( "../" )
Pkg.instantiate()

```

Activating project at `~/CODE/OpenDleto`

Once you have added Dleto to your Julia packages you can load the core packages for this tutorial: Plots and Dleto.

For beginner readability in later cells: - use `Dleto.direct_sum(...)` instead of typing the Unicode operator `⊗`; - use wrapper style `Dleto.ts(Y) * Xs` for reconstruction checks; - run `Dleto.warmup()` once per fresh kernel to precompile common tensor workflows.

PlotlyJS is optional. This notebook runs with plain Plots only.

```
[3]: using Plots: plot
      using Dleto
      using Dleto.TensorSpace: ts,
```

Loading Dleto KrylovKit Extension

WebIO._IJuliaInit()

Loading Dleto Plots Extension

Loading Dleto IterativeSolvers Extension

Loading Dleto IterativeSolvers Extension

1.1.4 Optional warmup (recommended once per fresh kernel)

The first call to major Dleto routines pays compilation cost. Run the next cell once after restarting the Julia kernel to precompile common tensor workflows and speed up later cells.

Use `Dleto.warmup()` for this, so tutorials stay consistent across labs.

```
[4]: Dleto.warmup()
```

Warming up Dleto kernels... (first run compiles methods)

Using SVDSolver...

Converting LinearMap to Matrix for SVD...

[Info: der: nullity 2 certified (rule :gap), cut at the Float64 precision floor 1.7763568394002505e-15 (relative) -- a near-derivation hidden below that floor would look identical. Values around the cut: [7.868341101001686e-17, 3.168058059975203e-16] | [2.6490879154535426e-6, 5.696911469745773e-6, 1.0054274071099565e-5]

[Info: Found 2 derivations for stratification.

Warmup complete.

```
[4]: (dims = (6, 5, 4), tol = 1.0e-6, eltype = Float64, stratify = true, nondeg = true)
```

Select your viewer properties, `:widescreen` for laptop & desktop viewing and `:vertical` for vertical screens like tablets/phones:

```
[5]: layout=(1,2); pic_size=(900, 400); set_compare_layout(:widescreen)
      # layout=(2,1); pic_size=(400, 900); set_compare_layout(:vertical)
```

1.1.5 1.2 Creating a Tensor Experiment

We create two tensors, one randomized for control, and one identical in size but with 2 rows, 2 columns, and 2 slices set to approximately zero. We then randomize the experiment tensor by applying a change in coordinates. We will demonstrate how to use chiseling to detect and recover the hidden zero rows / columns / slices.

Notation. Julia accepts $\LaTeX$ styled commands with tab-completion. For example, to insert the Unicode character for Γ use `\Gamma` in the code area followed by `tab` (or

cut-and-paste a character you see somewhere else). If you prefer, you can replace these characters with strings of characters of your choosing. `Dleto.jl` calculations make substantial use of tensors, matrices, and lists of matrices, so we recommend using a convention that clearly delineates roles.

We will use: * Greek capitals such as Γ (`\Gamma`), Δ (`\Delta`), Σ (`\Sigma`), Ξ (`\Xi`), Υ (`\Upsilon`) for tensors; * English capitals such as X , Y , Z for matrices; * Greek lower-case letters for real numbers; * English lower-case letters for integers; and * variables that suggest plurality, such as Γ s and X s, for lists.

Following Magma system implementations, since v1.1 the Dleto system has transitioned to its own internal tensor data type called `TensorSpace` (a generalization of `VectorSpace`). Any terms of this data types are called **tensors**. The function `ts(...)` is a convenience to convert Julia `AbstractArray` tensor data into the Dleto format.

```
[6]: ds = (7,6,5)
      Γ = ts(randn(Float64, ds))

      es = (5,5,5)
      Δ = ts(randn(Float64, es))   ts(zeros(Float64, ds .- es))
```

```
[6]: Dleto.TensorSpace.TensorElement(ITensor ord=3
Dim 1: (dim=7|id=441|"sum1")
Dim 2: (dim=6|id=33|"sum2")
Dim 3: (dim=5|id=993|"sum3")
NDTensors.Dense{Float64, Vector{Float64}}
7×6×5
[:, :, 1] =
 0.004497964892209621  -0.34315601458780676  ...  -0.8347277358401621  0.0
-0.46275081121338796  -0.1695454662558609          0.7228310845629764  0.0
 1.345146555608266    1.5036424604256218          0.2760370821388372  0.0
 0.816876701615627    -0.8324806479569866        -0.6910381233885684  0.0
-1.388410125469812    -1.1237958338109117          1.1411544641253235  0.0
 0.0                  0.0                  ...  0.0          0.0
 0.0                  0.0                  ...  0.0          0.0

[:, :, 2] =
-0.16391870434275824   0.5164491012604613  ...   0.021897146533723814  0.0
 0.17530005927202585  -0.1702064399741317          0.8214434679779871  0.0
-0.5720273616509783   -1.9942235734524256        -0.4690600911305835  0.0
 0.6086463865251738    0.7177694006747192          0.750699594090221   0.0
 0.9277706565431888    0.7404220969628008          1.1951747455721162  0.0
 0.0                  0.0                  ...  0.0          0.0
 0.0                  0.0                  ...  0.0          0.0

[:, :, 3] =
-1.2188228612690009   -1.0412430805244748  ...   0.35678619572257064  0.0
-0.800374251826328   -0.4266846058346815  ...  -0.1103460019631736  0.0
```

```

0.47211443989925617    1.6968431454895851    -0.692179234323266    0.0
-1.4860945488859285    -0.13175945344545778    1.2552060056152796    0.0
-1.840056841399681    -0.2945086705521898    -0.19929897928185583    0.0
0.0                    0.0                    ...    0.0                    0.0
0.0                    0.0                    ...    0.0                    0.0

```

```
[:, :, 4] =
```

```

-1.260228623141958    -2.3723143591273126    ...    -0.6364090215293995    0.0
-1.0574981865350446    0.37682708688148325    ...    -0.5366840183866037    0.0
1.0603804926154474    1.031738652262297    ...    0.7294116042514731    0.0
0.9408774840316516    -1.3254604833276722    ...    2.008895398687442    0.0
1.1790468509377487    -0.5103264464880197    ...    -0.5047821606860797    0.0
0.0                    0.0                    ...    0.0                    0.0
0.0                    0.0                    ...    0.0                    0.0

```

```
[:, :, 5] =
```

```

0.11122085676010855    -1.2000117326923092    ...    0.7137611323383445    0.0
1.4051558567956786    -1.7503158042645206    ...    0.7528448315146233    0.0
-1.1449157625594697    1.3463048031613067    ...    -0.5511800636283724    0.0
0.1777932531469167    0.662960728716191    ...    0.3479718805370421    0.0
1.0623244638293847    0.3462129138100546    ...    -0.9957859984224029    0.0
0.0                    0.0                    ...    0.0                    0.0
0.0                    0.0                    ...    0.0                    0.0

```

```
)
```

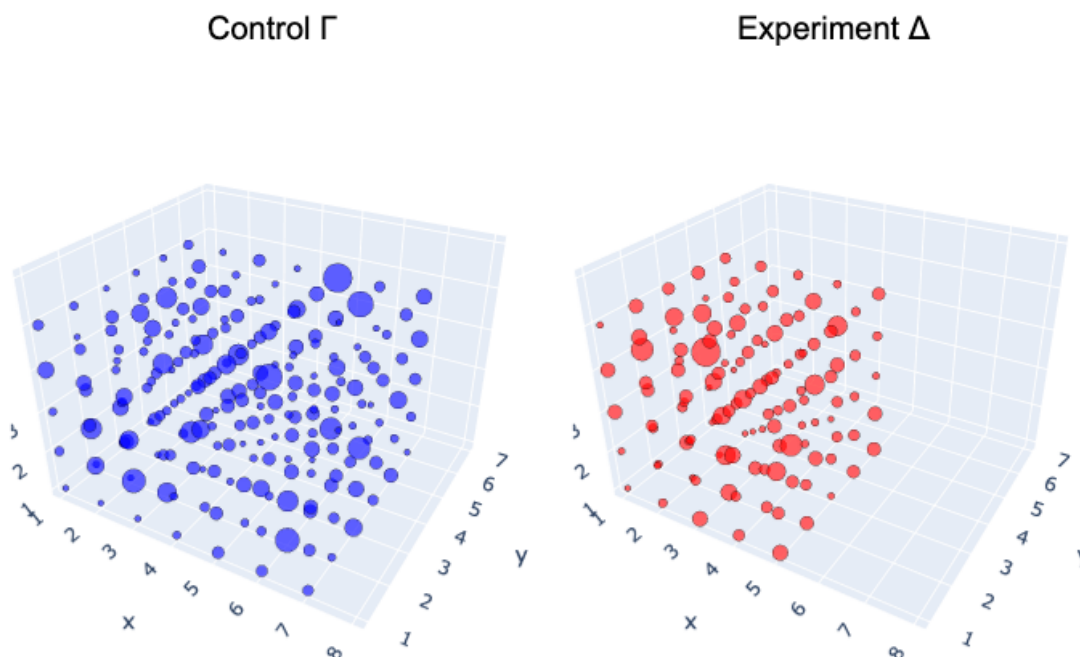
Visualization tools are very useful for larger tensors:

```

[7]: p1 = plot_tensor( $\Gamma$ ; title="Control  $\Gamma$ ", color=:blue)    # tol bounds how much
      ↪noise is shown
p2 = plot_tensor( $\Delta$ ; title="Experiment  $\Delta$ ", color=:red)
plot(p1, p2, layout=layout, size=pic_size) # Side-by-side plots

```

```
[7]:
```



The control tensor is unlikely to have large regions of zeros, while the experiment tensor is constructed so that it does. You will observe changes when you adjust the parameters.

1.1.6 1.3 Tensor Randomization

Dleto detects features such as redundancy without knowing about them in advance. We provide a demonstration. First, we scramble the experiment tensor to obscure its features. The scrambling process applies random basis changes to each of the original coordinate axes.

With the TensorSpace wrapper we state reconstruction checks explicitly as

$$\text{Dleto.ts}(\Gamma) * X_s \approx \Gamma_{\text{rand}}.$$

Dleto handles the internal tensor representation for these operations, so this notebook can stay focused on the algebra and interpretation.

```
[8]: Γ_rand, X1s = randomize_tensor(Γ)
    @assert Γ * X1s  Γ_rand

    Δ_rand, Y1s = randomize_tensor(Δ)
    @assert Δ * Y1s  Δ_rand
```

The control and the experiment tensor now look similarly unstructured:

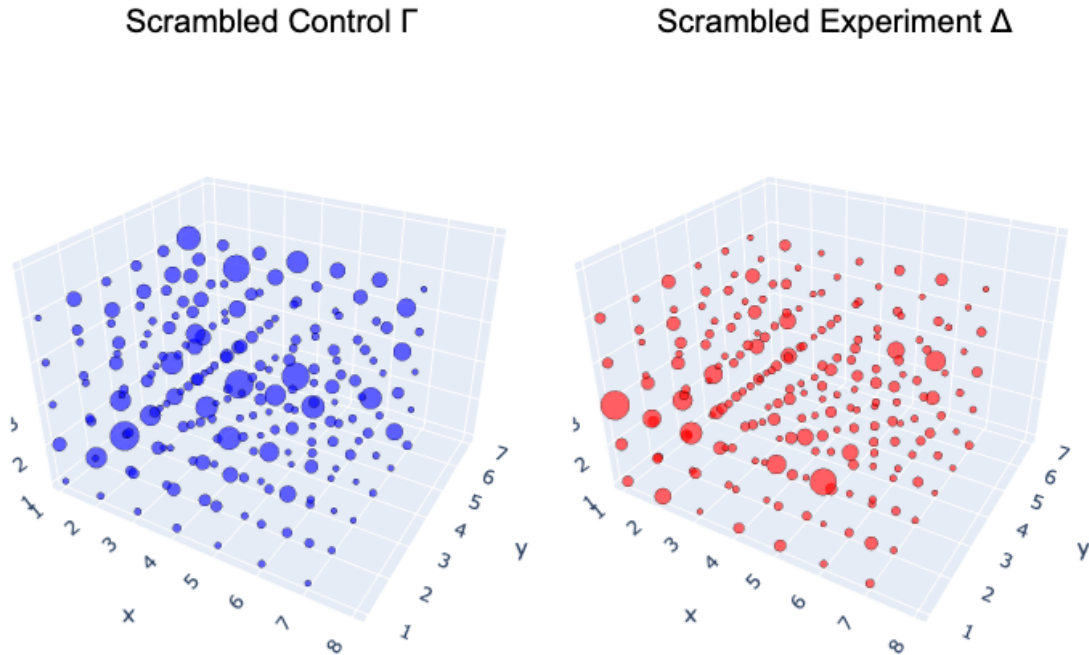
```
[9]: compare( $\Gamma$ _rand,  $\Delta$ _rand; left_title="Control  $\Gamma$ ", right_title="Experiment  $\Delta$ _rand")
```

Perhaps this is easier to see with a plot:

```
[10]: tol = 1e-5 # a small tolerance for numerical comparisons

p1 = plot_tensor( $\Gamma$ _rand, tol; title="Scrambled Control  $\Gamma$ ", color=:blue)
p2 = plot_tensor( $\Delta$ _rand, tol; title="Scrambled Experiment  $\Delta$ ", color=:red)
plot(p1, p2, layout=layout, size=pic_size)
```

[10]:



1.1.7 1.4 Chiseling for Tucker Decompositions

We now use Dleto to find a Tucker decomposition. The key command is `stratify`. While this function can be used with multiple parameters, it has default settings that detect structure when it is present and often perform well as a first approximation.

```
[11]:  $\Gamma$ _strat, X2s = stratify( $\Gamma$ _rand; tol=tol) # specify tol to match noise level
@assert  $\Gamma$ _rand * X2s  $\Gamma$ _strat
 $\Delta$ _strat, Y2s = stratify( $\Delta$ _rand; tol=tol)
@assert  $\Delta$ _rand * Y2s  $\Delta$ _strat
```

Using SVDSolver...

Converting LinearMap to Matrix for SVD...

Using SVDSolver...

[Info: Found 2 derivations for stratification.

Converting LinearMap to Matrix for SVD...

Using SVDSolver...

Converting LinearMap to Matrix for SVD...

[Info: Found 22 derivations for stratification.

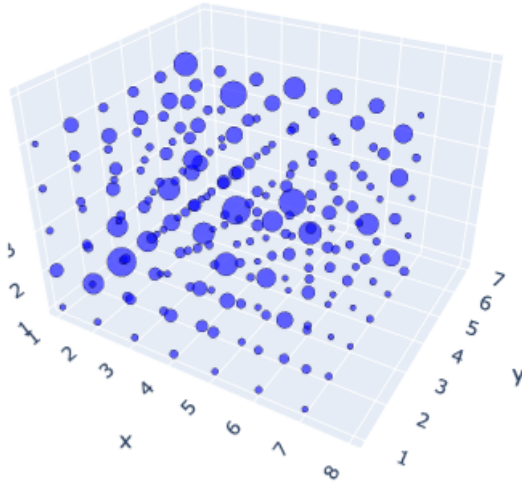
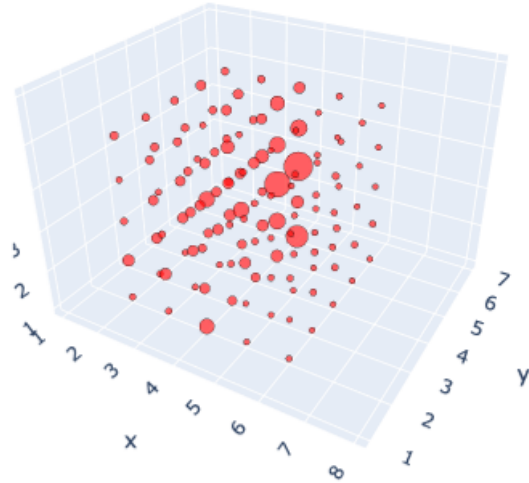
The output is a tensor-transform pair. You may observe some extra information printed that reports the total number “derivations” detected during stratification. Random dense tensors typically admit only 2 derivations. When a Tucker decomposition exists in a tensor, on the other hand, you will typically see more than 2 derivations. Indeed you can think of a derivation number greater than 2 as a proxy for Dleto having detected something of interest in your tensor.

Round 2? If you modified the parameters you may observe that stratification no longer distinguishes the experiment tensor. You now need to consider how much tolerance to pass along to the stratification by adding the optional parameter `tol=tol` (the tolerance we set above). Replace the original commands by the commented commands to include this tolerance adjustment. You will know that your tolerance is adequate if the number derivations for the experiment is more than 2.

Let us look at the results:

```
[12]: p1 = plot_tensor( $\Gamma$ _strat, tol; title="Stratified Control  $\Gamma$ ", color=:blue)
      p2 = plot_tensor( $\Delta$ _strat, tol; title="Stratified Experiment  $\Delta$ ", color=:red)
      plot(p1, p2, layout=layout, size=pic_size)
```

[12]:

Stratified Control Γ Stratified Experiment Δ 

If the noise-to-dimension ratio is reasonable, you should observe that the experiment tensor now clusters nearly all its values in one region. Although the location of that region may shift from experiment to experiment, it should have similar volume to the initial nonzero region in the experiment tensor.

This is a good place to return to the start of this section and begin altering the parameters to see how the results change.

* First you could increase the dimension. **Caution:** the complexity of a tensor is proportional to the volume. Changing (5,5,5) to (25,25,25) increases the input by a factor of 125, not 5! You should therefore proceed with caution until you acclimate to the impact your changes have on performance and memory. * Secondly, you could increase the background noise and set the tolerance to match. For example, `tol=2*db` is an appropriate range.

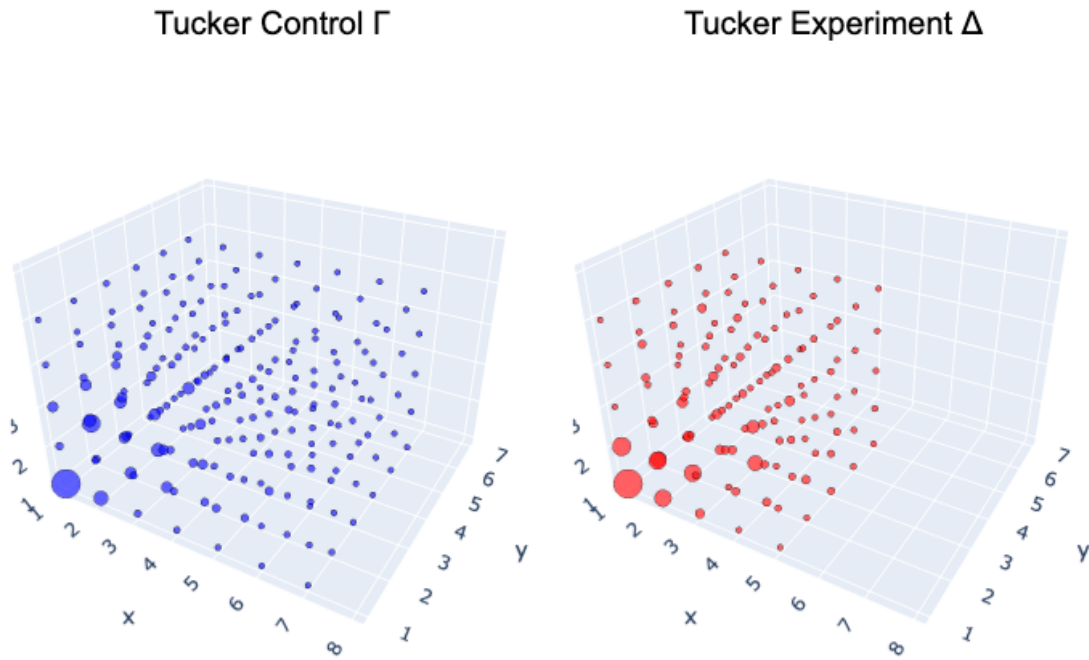
1.1.8 1.5 Optimal Tucker Decompositions

As noted earlier, Dleto has a function `nondeg` to perform Tucker computations more efficiently. The function accepts two modes `:full` to regroup the nonzero values inside the original space, and `:trunc` to truncate the detected zero rows, columns, slices. It is usually advisable to pass to a nondegenerate tensor using `nondeg` at the start of any computation because it has a relatively low cost and the dimension of the resulting tensor can be (sometimes quite substantially) smaller. This can reduce the cost of all subsequent computations.

```
[13]:  $\Gamma_{\text{nondeg}}, X3s = \text{Dleto.nondeg}(\Gamma_{\text{rand}}, \text{mode}=:full);$  # use mode=:trunc to
      ↪ truncate the 0's in the Tucker decomposition
      @assert  $\Gamma_{\text{rand}} * X3s \approx \Gamma_{\text{nondeg}}$ 
       $\Delta_{\text{nondeg}}, Y3s = \text{Dleto.nondeg}(\Delta_{\text{rand}}, \text{mode}=:full);$ 
      @assert  $\Delta_{\text{rand}} * Y3s \approx \Delta_{\text{nondeg}}$ 

      p1 = plot_tensor( $\Gamma_{\text{nondeg}}$ , tol; title="Tucker Control  $\Gamma$ ", color=:blue)
      p2 = plot_tensor( $\Delta_{\text{nondeg}}$ , tol; title="Tucker Experiment  $\Delta$ ", color=:red)
      plot(p1, p2, layout=layout, size=pic_size)
```

[13]:



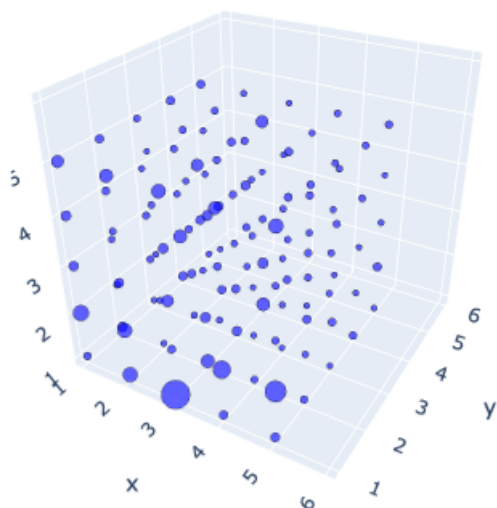
```
[14]:  $\Delta_{\text{nondeg}}, Y4s = \text{Dleto.nondeg}(\Delta_{\text{rand}});$ 
      @assert  $\Delta_{\text{rand}} * Y4s \approx \Delta_{\text{nondeg}}$ 

       $\Delta_{\text{nondeg\_trunc}}, Y4s = \text{Dleto.nondeg}(\Delta_{\text{rand}}, \text{mode}=:trunc);$ 
      @assert  $\Delta_{\text{rand}} * Y4s \approx \Delta_{\text{nondeg\_trunc}}$ 

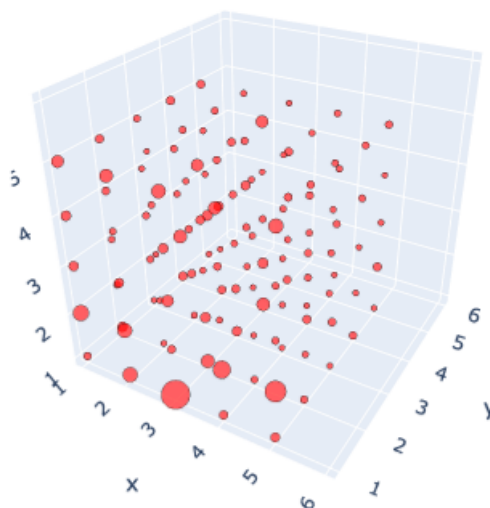
      p1 = plot_tensor( $\Delta_{\text{nondeg}}$ , tol; title="Tucker Experiment  $\Delta$  full", color=:blue)
      p2 = plot_tensor( $\Delta_{\text{nondeg\_trunc}}$ , tol; title="Tucker Experiment  $\Delta$  truncated",
      ↪ color=:red)
      plot(p1, p2, layout=layout, size=pic_size)
```

[14]:

Tucker Experiment Δ full



Tucker Experiment Δ truncated



Tucker decompositions are closures: repeated applications achieve no further gains (though it may reorder and rescale the one its has already carried out):

```
[15]: Δ_nondeg_trunc, _ = nondeg(Δ_rand, mode=:trunc);
      Δ_nondeg2_trunc, _ = nondeg(Δ_nondeg_trunc, mode=:trunc);
      @assert size(Δ_nondeg_trunc) == size(Δ_nondeg2_trunc)
```

If you did not already change parameters above, we invite you to return to the first cell and do so now. We can continue to other experiments when you are ready!

1.1.9 (Optional) Carry out the trash

While it is helpful to visualize the effects of Dleto chiseling, these 3D plots (especially of large dense randomized data) can use up much the available memory of a notebook environment. If you have limited resources to run the notebook or experience lag times then you may benefit from running the following garbage collection routines between sections. If you are very tight on memory budgets you may wish to clear output data of the 3D plots above.

```
[16]: # release memory of tensors
      Γ=nothing; Δ=nothing
      Γ_rand=nothing; Δ_rand=nothing
```

```

Γ_strat=nothing; Δ_strat=nothing
X1s=nothing; Y1s=nothing
X2s=nothing; Y2s=nothing

# release memory plots. Note, in some Jupyter environments, the 3D plots will
↳ persist as an image but no longer be interactive.
p1=nothing
p2=nothing
GC.gc()

```

2 2. Block Diagonalization

You can think of performing a Tucker decomposition as finding a single block on the diagonal of a tensor. It is of course natural to ask whether one can find several distinct blocks (assuming they exist). In similar fashion to the first experiment, we now create such a tensor and hide the blocks to see if Dleto chiseling can recover them.

```

[17]: as=[3,2,7]
      bs=[4,5,2]
      cs=[5,3,3]
      ds=as+bs+cs

      Γ = ts(randn(ds...)); # a control tensor

      Δ = ts(randn(as...))  ts(randn(bs...))  ts(randn(cs...))

      # Add noise to experiment tensor
      # db = 0.001; tol = 10*db          # Add some noise, increase tolerance
      # Δ += db*randn(size(Δ)...)

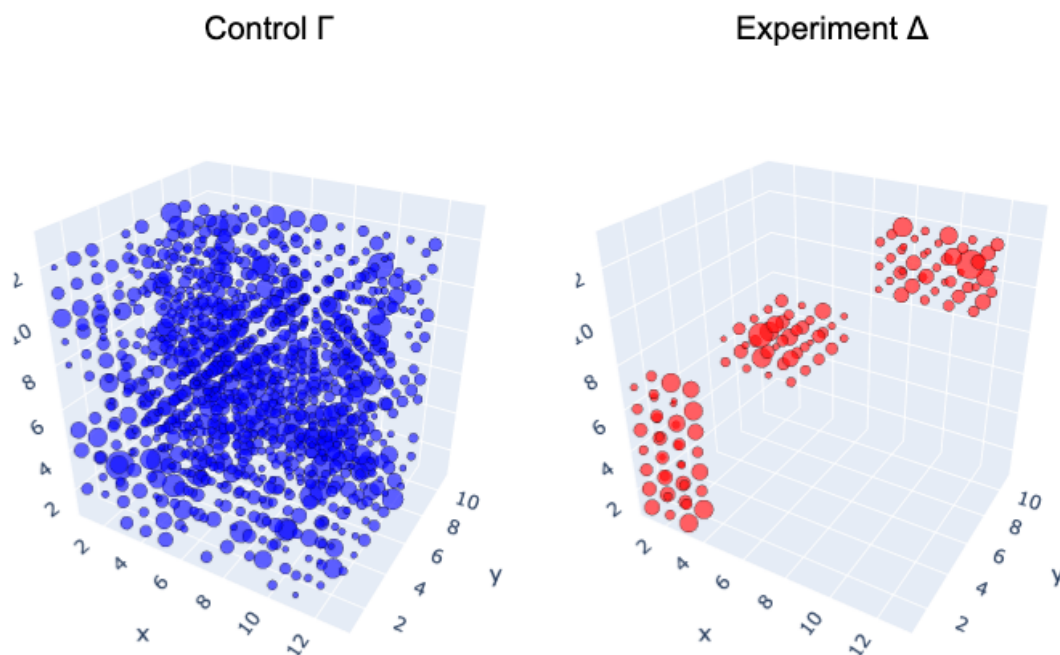
      p1 = plot_tensor(Γ; title="Control Γ", color=:blue)
      p2 = plot_tensor(Δ; title="Experiment Δ", color=:red)
      plot(p1, p2, layout=layout, size=pic_size)

```

Warning: More than 1,000 viewable
entries, plotting only the largest points.
@ DletoPlotsExt

~/CODE/OpenDleto/ext/DletoPlotsExt.jl:264

[17]:



We again hide the structure by randomizing the reference frame of our tensor:

```
[18]: Γ_rand, X1s = randomize_tensor(Γ);
      Δ_rand, Y1s = randomize_tensor(Δ); # In theory Δ_rand = Δ*Xs

      # Plotted these two tensors are essentially indistinguishable from each other.
      p1 = plot_tensor(Γ_rand; title="Scrambled Control Γ", color=:blue)
      p2 = plot_tensor(Δ_rand; title="Scrambled Experiment Δ", color=:red)
      plot(p1, p2, layout=layout, size=pic_size)
```

Warning: More than 1,000 viewable entries, plotting only the largest points.
@ DletoPlotsExt

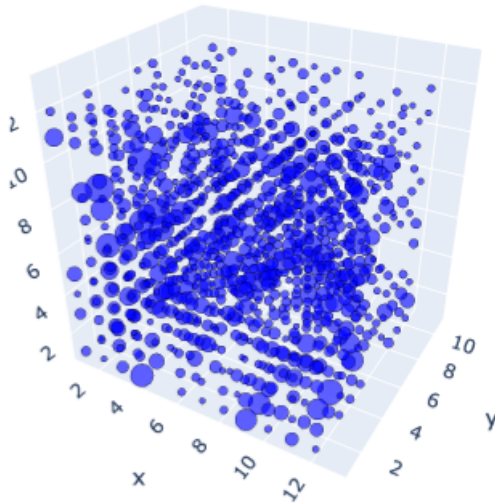
~/CODE/OpenDleto/ext/DletoPlotsExt.jl:264

Warning: More than 1,000 viewable entries, plotting only the largest points.
@ DletoPlotsExt

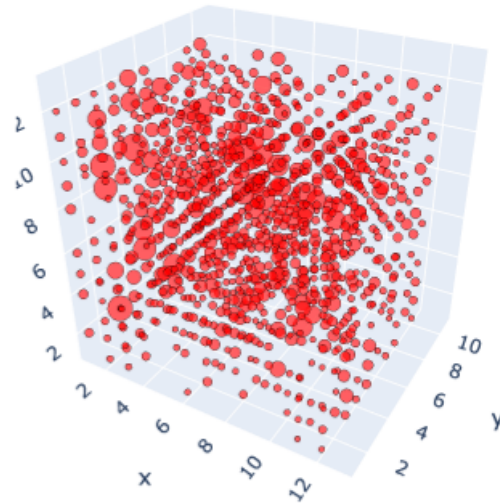
~/CODE/OpenDleto/ext/DletoPlotsExt.jl:264

[18]:

Scrambled Control Γ



Scrambled Experiment Δ



This at least provides visual assurance that we have hidden the block structure in our experiment tensor. Let us now try to use chiseling to recover it:

```
[19]: # If you added noise a couple blocks up, then adjust this tolerance to match
      ↪ your noise
      # tol = 1e-3 # specify tolerance for stratification
      Γ_strat, X2s = stratify(Γ_rand; tol=tol) # specify tol to match noise level
      @assert Γ_rand * X2s ≈ Γ_strat
      Δ_strat, Y2s = stratify(Δ_rand; tol=tol)
      @assert Δ_rand * Y2s ≈ Δ_strat
```

Using SVDSolver...

Converting LinearMap to Matrix for SVD...

Using SVDSolver...

Converting LinearMap to Matrix for SVD...

Using SVDSolver...

[Info: Found 2 derivations for stratification.

Converting LinearMap to Matrix for SVD...

Using SVDSolver...

Converting LinearMap to Matrix for SVD...

[Info: Found 32 derivations for stratification.

The volume of the dots are proportional to the scalars they represent. In particular, tiny values are omitted entirely from the plots. Inspecting the actual data provides more detail. Up a small tolerance we see that we have recovered the degeneracy embedded in the experimental tensor.

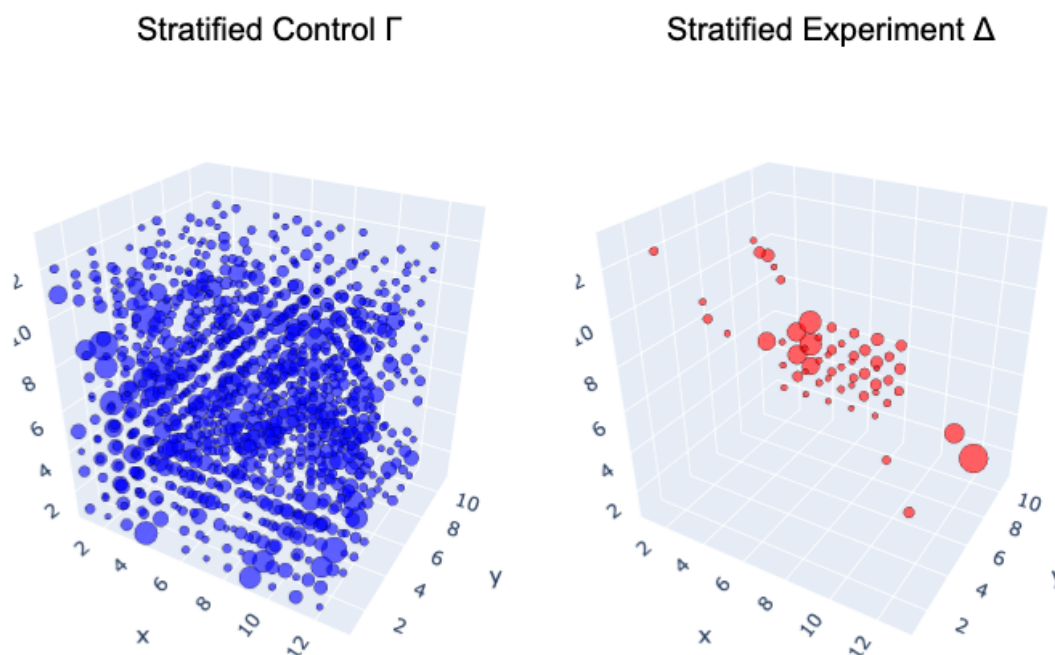
```
[20]: p1 = plot_tensor( $\Gamma$ _strat, tol; title="Stratified Control  $\Gamma$ ", color=:blue)
      p2 = plot_tensor( $\Delta$ _strat, tol; title="Stratified Experiment  $\Delta$ ", color=:red)
      plot(p1, p2, layout=layout, size=pic_size)
```

Warning: More than 1,000 viewable entries, plotting only the largest points.

@ DletoPlotsExt

~/CODE/OpenDleto/ext/DletoPlotsExt.jl:264

[20]:



Once more we have recovered some structure. The order of our blocks has changed (and it may be helpful to interact with 3D plot to understand the structure).

We again invite you to revisit the experiment and add noise, add more blocks, and change dimensions.

2.0.1 Carry out the trash

If you are ready for the next experiment you might get better viewing results if you release some more trash.

```
[21]: # release memory of tensors
Γ =nothing; Δ=nothing
Γ_rand=nothing; Δ_rand=nothing
Γ_strat=nothing; Δ_strat=nothing
X1s=nothing; Y1s=nothing
X2s=nothing; Y2s=nothing

# release memory plots. Note, in some Jupyter environments, the 3D plots will
↳ persist as an image but no longer be interactive.
p1=nothing
p2=nothing
GC.gc()
```

2.1 3 Stratification

In linear algebra there are many uses for diagonals, block diagonals, and triangular forms of matrices. Moving from matrices to tensors with more than two axes, we encounter something new and unrelated to matrices: curves and surfaces.

In keeping with previous demonstrations, we carry out a controlled experiment.

```
[22]: ds = (30,30,30)
Γ = ts(randn(Float64, ds...)) # a control tensor

db = 0.0
tol = 1e-4

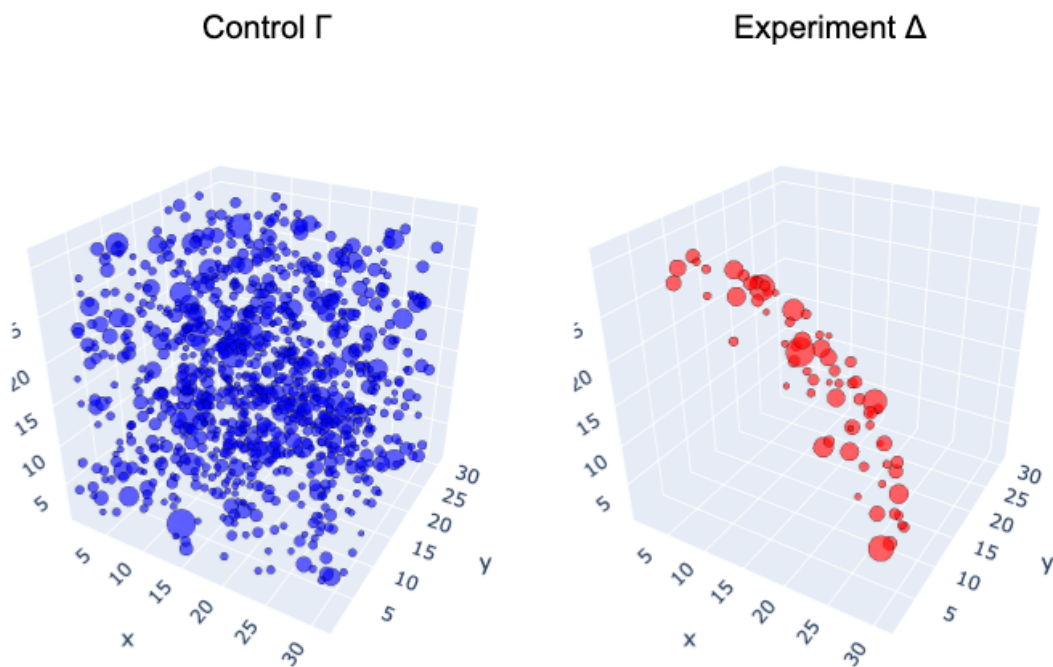
# An experiment tensor with a hidden surface.
# - Choose f(x,y,z) to define the surface
f(x,y,z) = x^2 + y^2 + z^2
# - Populate Δ near the surface defined by f
Δ = ts(zeros(Float64, ds...));
for i in 1:ds[1], j in 1:ds[2], k in 1:ds[3]
    if abs( f(i,j,k) - 30^2 ) < 1 + 2*randn()
        Δ[i,j,k] = randn()
    end
end

# to avoid long wait times raise the threshold for plotting
p1 = plot_tensor(Γ; title="Control Γ", color=:blue)
p2 = plot_tensor(Δ; title="Experiment Δ", color=:red)
plot(p1, p2, layout=layout, size=pic_size)
```

Warning: More than 1,000 viewable entries, plotting only the largest points.
 @ DletoPlotsExt

~/CODE/OpenDleto/ext/DletoPlotsExt.jl:264

[22]:



Dleto supplies a range of convenient functions to generate tensors of this sort. It also provides configurable noise and distance functions.

```
[23]: Γ_rand, X1s = randomize_tensor(Γ)
      @assert Γ * X1s  Γ_rand
      Δ_rand, Y1s = randomize_tensor(Δ)
      @assert Δ * Y1s  Δ_rand
```

The randomized versions are by now visually familiar:

```
[24]: p1 = plot_tensor(Γ_rand; title="Scrambled Control Γ", color=:blue)
      p2 = plot_tensor(Δ_rand; title="Scrambled Experiment Δ", color=:red)
      plot(p1, p2, layout=layout, size=pic_size)
```

Warning: More than 1,000 viewable entries, plotting only the largest points.
 @ DletoPlotsExt

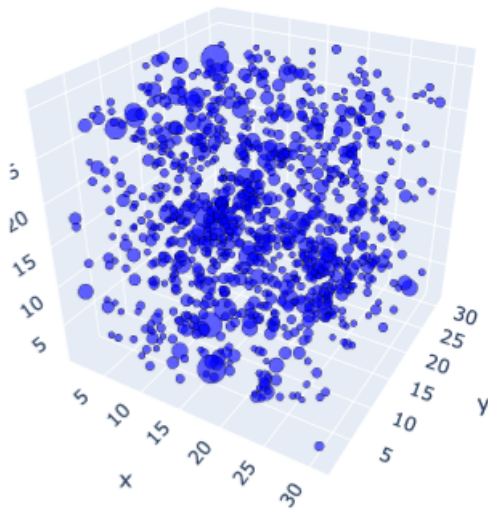
~/CODE/OpenDleto/ext/DletoPlotsExt.jl:264

Warning: More than 1,000 viewable entries, plotting only the largest points.
 @ DletoPlotsExt

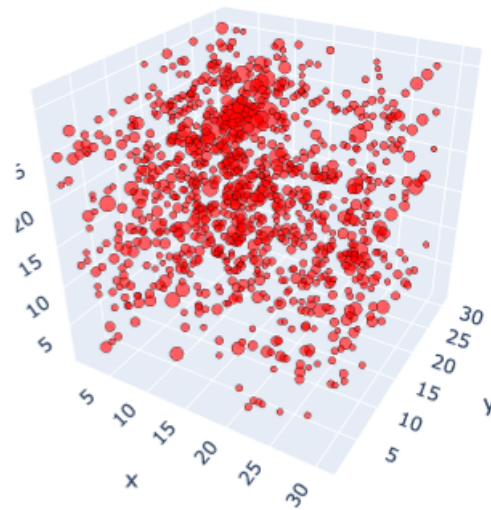
~/CODE/OpenDleto/ext/DletoPlotsExt.jl:264

[24]:

Scrambled Control Γ



Scrambled Experiment Δ



```
[25]: @time Γ_strat, X2s = stratify(Γ_rand; tol=1e-2)    # specify tol to match noise
      ↪ level
      @assert Γ_rand * X2s    Γ_strat
```

Using SVDSolver...

Converting LinearMap to Matrix for SVD...

[Info: Found 2 derivations for stratification.

3.752340 seconds (22.89 M allocations: 1.255 GiB, 2.93% gc time, 91.84% compilation time)

Since we are trying to stratify larger surfaces, it makes sense (as we suggested earlier) to first remove any degeneracies:

```
[26]: Δ_nondeg, Y2s = Dleto.nondeg(Δ_rand, mode=:trunc);
      @assert Δ_rand * Y2s    Δ_nondeg
      @time Δ_strat, Z1s = stratify(Δ_nondeg; tol=1e-1)
```

```
@assert Δ_nondeg * Z1s Δ_strat
```

Using SVDSolver...

Converting LinearMap to Matrix for SVD...

Using SVDSolver...

Converting LinearMap to Matrix for SVD...

[Info: Found 18 derivations for stratification.

0.553199 seconds (226.71 k allocations: 159.945 MiB, 6.43% gc time, 11.40% compilation time)

```
[27]: p1 = plot_tensor(Γ_strat; title="Stratified Control Γ", color=:blue)
      p2 = plot_tensor(Δ_strat; title="Stratified Experiment Δ", color=:red)
      plot(p1, p2, layout=layout, size=pic_size)
```

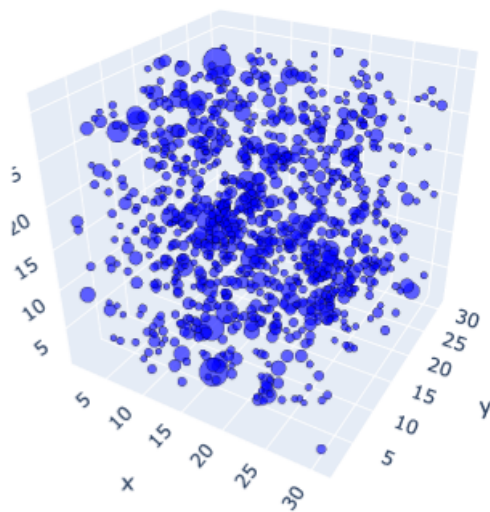
Warning: More than 1,000 viewable entries, plotting only the largest points.

@ DletoPlotsExt

~/CODE/OpenDleto/ext/DletoPlotsExt.jl:264

[27]:

Stratified Control Γ



Stratified Experiment Δ

